Vector Semantics and Embeddings

Thanks to Dr. Ashraf Uddin Sir

Vector Semantics and Embeddings

Words that occur in similar contexts tend to have similar meanings.

Vector semantics instantiates learning representations of the meaning of words, called embeddings, directly from their distributions in texts.

These representations are used in every natural language processing application that makes use of meaning, and the **static embeddings** we introduce here underlie the more powerful **dynamic** or **contextualized embeddings** like BERT.

Vector Semantics

Vectors semantics is the standard way to represent word meaning in NLP, helping us model many of the aspects of word meaning .

For example, suppose you didn't know the meaning of the word **ongchoi** (a recent borrowing from Cantonese) but you see it in the following contexts:

(6.1) Ongchoi is delicious sauteed with garlic.
(6.2) Ongchoi is superb over rice.
(6.3) ...ongchoi leaves with salty sauces...

And suppose that you had seen many of these context words in other contexts:

(6.4) ...spinach sauteed with garlic over rice...
(6.5) ...chard stems and leaves are delicious...
(6.6) ...collard greens and other salty leafy greens

Vector Semantics

The fact that **ongchoi** occurs with words like **rice** and **garlic** and **delicious** and **salty**, as do words like **spinach**, **chard**, and **collard greens** might suggest that **ongchoi** is a leafy green similar to these other leafy greens. We can do the same thing computationally by just counting words in the context of **ongchoi**

(6.1) Ongchoi is delicious sauteed with garlic.
(6.2) Ongchoi is superb over rice.
(6.3) ...ongchoi leaves with salty sauces...

And suppose that you had seen many of these context words in other contexts:

(6.4) ...spinach sauteed with garlic over rice...
(6.5) ...chard stems and leaves are delicious...
(6.6) ...collard greens and other salty leafy greens

Vector Semantics

The idea of vector semantics is to represent a word as a point in a multidimensional semantic space that is derived from the distributions of word neighbors. Vectors for representing words are called embeddings (more strictly applied only to dense vectors like word2vec).

A two-dimensional projection of embeddings for some words and phrases, showing that words with similar meanings are nearby in space. The original 60-dimensional embeddings were trained for sentiment analysis.

Vector Semantics

The fine-grained model of word similarity of vector semantics offers enormous power to NLP applications.

NLP applications like the sentiment classifiers depend on the same words appearing in the training and test sets. But by representing words as embeddings, classifiers can assign sentiment as long as it sees some words with similar meanings.

And as we'll see, vector semantic models can be learned automatically from text without supervision.

Vector Semantics

We'll introduce the two most commonly used models. In the **tf-idf** model, an important baseline, the meaning of a word is defined by a simple function of the counts of nearby words. We will see that this method results in very **long vectors that are sparse**, i.e. mostly zeros (since most words simply never occur in the context of others).

We'll introduce the **word2vec** model family for constructing short, **dense vectors** that have useful semantic properties. We'll also introduce the cosine, the standard way to use embeddings to compute semantic similarity, between two words, two sentences, or two documents, an important tool in practical applications like question answering, and summarization.

Words and Vectors

Vector or distributional models of meaning are generally based on a **co-occurrence matrix**, a way of representing how often words co-occur. We'll look at two popular matrices: the **term-document** matrix and the **term-term** matrix.

In a term-document matrix, each row represents a word in the vocabulary and each column represents a document from some collection of documents.

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

Figure 6.2 The term-document matrix for four words in four Shakespeare plays. Each cell contains the number of times the (row) word occurs in the (column) document.

Words and Vectors

We can think of the vector for a document as a point in |V|-dimensional space;

Thus, the documents in Fig. 6.3 are points in 4-dimensional space.

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

Figure 6.3 The term-document matrix for four words in four Shakespeare plays. The red boxes show that each document is represented as a column vector of length four.

A real term-document matrix, of course, wouldn't just have 4 rows and columns. More generally, the term-document matrix has |V| rows (one for each word type in the vocabulary) and D columns (one for each document in the collection.

Word as vector

We've seen that documents can be represented as vectors in a vector space. But vector semantics can also be used to represent the meaning of words. We do this row vector by associating each word with a word vector— a row vector rather than a column vector, hence with different dimensions, as shown in Fig

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

Figure 6.5 The term-document matrix for four words in four Shakespeare plays. The red boxes show that each word is represented as a row vector of length four.

Word as vector

An alternative to using the term-document matrix to represent words as vectors of document counts, is to use the **term-term matrix**, also called the word-word matrix or the **term-context matrix**, in which the columns are labeled by words rather than documents. This matrix is thus of dimensionality **|V|×|V|**

The context could be the document, in which case the cell represents the number of times the two words appear in the same document. It is most common, however, to use smaller contexts, generally a **window** around the word, for example of 4 words to the left and 4 words to the right, in which case the cell represents the number of times (in some training corpus) the column word occurs in such a ±4 word window around the row word.

Word as vector

If we take every occurrence of each word (say strawberry) and count the context words around it, we get a word-word co-occurrence matrix.

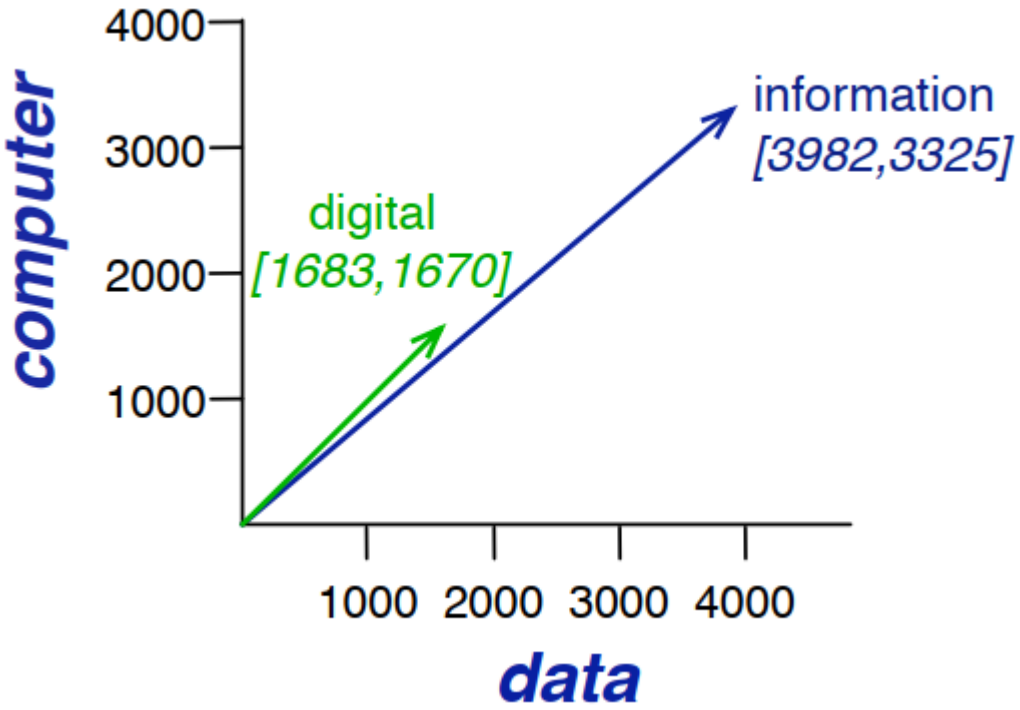
Note in Fig. 6.6 that the two words cherry and strawberry are more similar to each other (both pie and sugar tend to occur in their window) than they are to other words like digital; conversely, digital and information are more similar to each other than, say, to strawberry.

	aardvark	...	computer	data	result	pie	sugar	...
cherry	0	...	2	8	9	442	25	...
strawberry	0	...	0	0	1	60	19	...
digital	0	...	1670	1683	85	5	4	...
information	0	...	3325	3982	378	5	13	...

Figure 6.6 Co-occurrence vectors for four words in the Wikipedia corpus, showing six of the dimensions (hand-picked for pedagogical purposes). The vector for *digital* is outlined in red. Note that a real vector would have vastly more dimensions and thus be much sparser.

Word as vector

A spatial visualization of word vectors for digital and information, showing just two of the dimensions, corresponding to the words data and computer.



Cosine for measuring similarity

To measure similarity between two target words **v** and **w**, we need a metric that takes two vectors (of the same dimensionality, either both with words as dimensions, hence of length **|V|**, or both with documents as dimensions as documents, of length **|D|**) and gives a measure of their similarity.

By far the most common similarity metric is the **cosine** of the angle between the vectors.

$$\text{cosine}(\mathbf{v},\mathbf{w}) = \frac{\mathbf{v}\cdot\mathbf{w}}{|\mathbf{v}||\mathbf{w}|} = \frac{\sum_{i=1}^{N} v_i w_i}{\sqrt{\sum_{i=1}^{N} v_i^2}\sqrt{\sum_{i=1}^{N} w_i^2}}$$

Cosine for measuring similarity

Let's see how the cosine computes which of the words cherry or digital is closer in meaning to information, just using raw counts from the following shortened table:

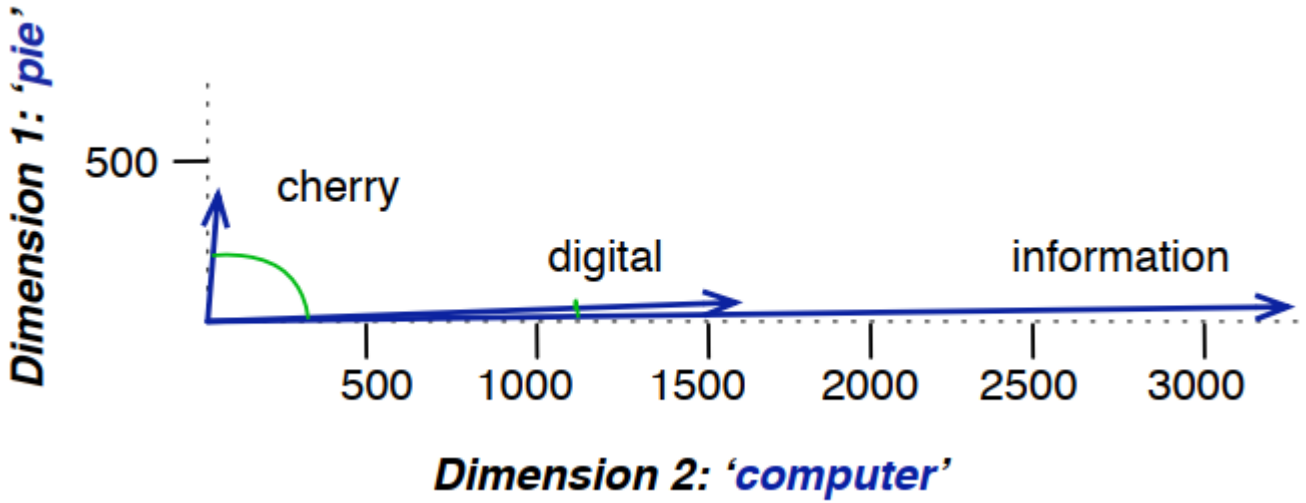
	pie	data	computer
cherry	442	8	2
digital	5	1683	1670
information	5	3982	3325

$$\cos(\text{cherry}, \text{information}) = \frac{442*5+8*3982+2*3325}{\sqrt{442^2+8^2+2^2}\sqrt{5^2+3982^2+3325^2}} = .018$$

$$\cos(\text{digital}, \text{information}) = \frac{5*5+1683*3982+1670*3325}{\sqrt{5^2+1683^2+1670^2}\sqrt{5^2+3982^2+3325^2}} = .996$$

Cosine for measuring similarity

A (rough) graphical demonstration of cosine similarity, showing vectors for three words (**cherry, digital, and information**) in the two dimensional space defined by counts of the words **computer** and **pie** nearby.



TF-IDF: Weighing terms in the vector

The co-occurrence matrix represents each cell by frequencies, either of words with documents, or words with other words. But raw frequency is not the best measure of association between words. Raw frequency is very skewed and not very **discriminative**.

The **tf-idf** weighting is the product of two terms, each term capturing one of these two intuitions:

The first is the **term frequency**: the frequency of the word **t** in the document **d**.

We can just use the raw count as the term frequency:

$$\text{tf}_{t,d} = \text{count}(t,d)$$
$$\text{tf}_{t,d} = \log_{10}(\text{count}(t,d)+1)$$

TF-IDF: Weighing terms in the vector

The second factor in **tf-idf** is used to give a higher weight to words that occur only in a few documents. Terms that are limited to a few documents are useful for discriminating those documents from the rest of the collection; terms that occur frequently across the entire collection aren't as helpful. The document frequency $\boldsymbol{df_t}$ of a term **t** is the number of documents it occurs in.

We emphasize discriminative words via the inverse document frequency or **idf** term weight. The idf is defined using the fraction: $\frac{\boldsymbol{N}}{\boldsymbol{df_t}}$ where **N** is the total number of documents in the collection.

$$\text{idf}_t = \log_{10}\left(\frac{N}{\text{df}_t}\right)$$

TF-IDF: Weighing terms in the vector

The tf-idf weighted value $\boldsymbol{w_{t,d}}$ for word **t** in document **d** thus combines term frequency $\boldsymbol{tf_{t,d}}$ with idf

$$w_{t,d} = \text{tf}_{t,d} \times \text{idf}_t$$

The tf-idf weighted value $\boldsymbol{w_{t,d}}$ for word **t** in document **d** thus combines term frequency $\boldsymbol{tf_{t,d}}$ with idf

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	0.074	0	0.22	0.28
good	0	0	0	0
fool	0.019	0.021	0.0036	0.0083
wit	0.049	0.044	0.018	0.022

the **0.049** value for wit in **As You Like It** is the product of tf = $log_{10}(20 + 1)$ = 1.322 and idf = 0.037.

Document vector

The tf-idf model of meaning is often used for document functions like deciding if two documents are similar. We represent a document by taking the vectors of all the words in the document, and computing the **centroid** of all those vectors.

The centroid is the multidimensional version of the mean; the centroid of a set of vectors is a single vector that has the minimum sum of squared distances to each of the vectors in the set. Given k word vectors w1;w2;:::;wk, the centroid document vector d is:

$$d = \frac{w_1 + w_2 + \ldots + w_k}{k}$$

Word2vec

We saw how to represent a word as a **sparse**, long vector with dimensions corresponding to words in the vocabulary or documents in a collection.

We now introduce a more powerful word representation: **embeddings**, short **dense vectors**.

It turns out that dense vectors work better in every NLP task than sparse vectors.

Representing words as 300-dimensional dense vectors requires our classifiers to learn far fewer weights than if we represented words as 50,000-dimensional vectors, and the smaller parameter space possibly helps with generalization and avoiding overfitting.

Word2vec

Dense vectors may also do a better job of capturing **synonymy**.

For example, in a sparse vector representation, dimensions for synonyms like **car** and **automobile** dimension are distinct and unrelated; **sparse vectors** may thus fail to capture the similarity between a word with **car** as a neighbor and a word with **automobile** as a neighbor.

We introduce one method for computing embeddings: skip-gram with negative sampling, sometimes called **SGNS**. The skip-gram algorithm is one of two algorithms in a software package called word2vec, and so sometimes the algorithm is loosely referred to as **word2vec**

Word2vec

The word2vec methods are fast, efficient to train, and easily available online with code and pretrained embeddings.

Word2vec embeddings are static beddings, meaning that the method learns one fixed embedding for each word in the vocabulary.

We'll introduce methods for learning dynamic contextual embeddings like the popular family of **BERT** representations, in which the vector for each word is different in different contexts.

The intuition of word2vec is that instead of counting how often each word **w** occurs near, say, **apricot**, we'll instead train a classifier on a binary prediction task: "Is word **w** likely to show up near **apricot**?"

Word2vec

word2vec is a much simpler model than the **neural network** language model, in two ways.

First, word2vec simplifies the task (making it binary classification instead of word prediction). Second, word2vec simplifies the architecture (training a logistic regression classifier instead of a multi-layer neural network with hidden layers). The intuition of skip-gram is:

1. Treat the target word and a neighboring context word as positive examples.
2. Randomly sample other words in the lexicon to get negative samples.
3. Use logistic regression to train a classifier to distinguish those two cases.
4. Use the learned weights as the embeddings.

Learning skip-gram embeddings

The learning algorithm for skip-gram embeddings takes as input a corpus of text, and a chosen vocabulary size **N**. It begins by assigning a random embedding vector for each of the **N** vocabulary words, and then proceeds to iteratively shift the embedding of each word **w** to be more like the embeddings of words that occur nearby in texts, and less like the embeddings of words that don't occur nearby. Let's start by considering a single piece of training data:

```
... lemon,  a [tablespoon of apricot jam,      a] pinch ...
               c1         c2    w     c3       c4
```

Learning skip-gram embeddings

```
... lemon,  a [tablespoon of apricot jam,      a] pinch ...
               c1         c2    w     c3       c4
```

This example has a target word **w** (apricot), and 4 context words in the **L** = ±2 window, resulting in 4 positive training instances (on the left below):

positive examples +

w	c_{pos}
apricot	tablespoon
apricot	of
apricot	jam
apricot	a

negative examples -

w	c_{neg}	w	c_{neg}
apricot	aardvark	apricot	seven
apricot	my	apricot	forever
apricot	where	apricot	dear
apricot	coaxial	apricot	if

Other kinds of static embeddings

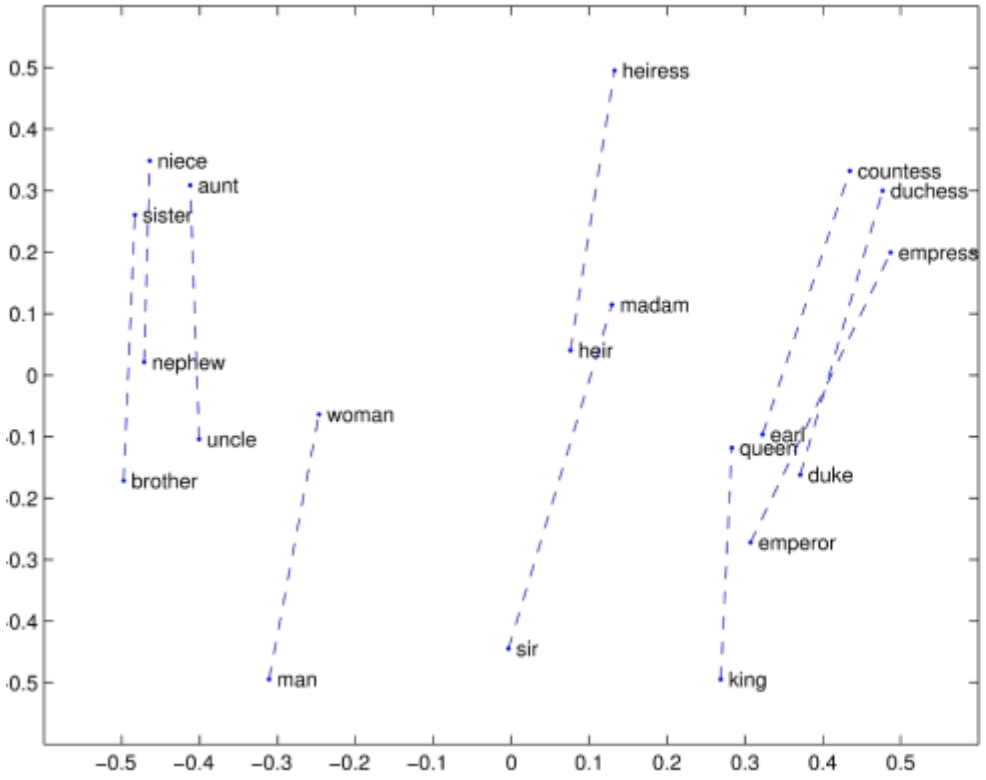
There are many kinds of **static embeddings**. An extension of **word2vec**, **fasttext**, addresses a problem with word2vec as we have presented it so far: it has no good way to deal with **unknown words**—words that appear in a test corpus but were unseen in the training corpus.

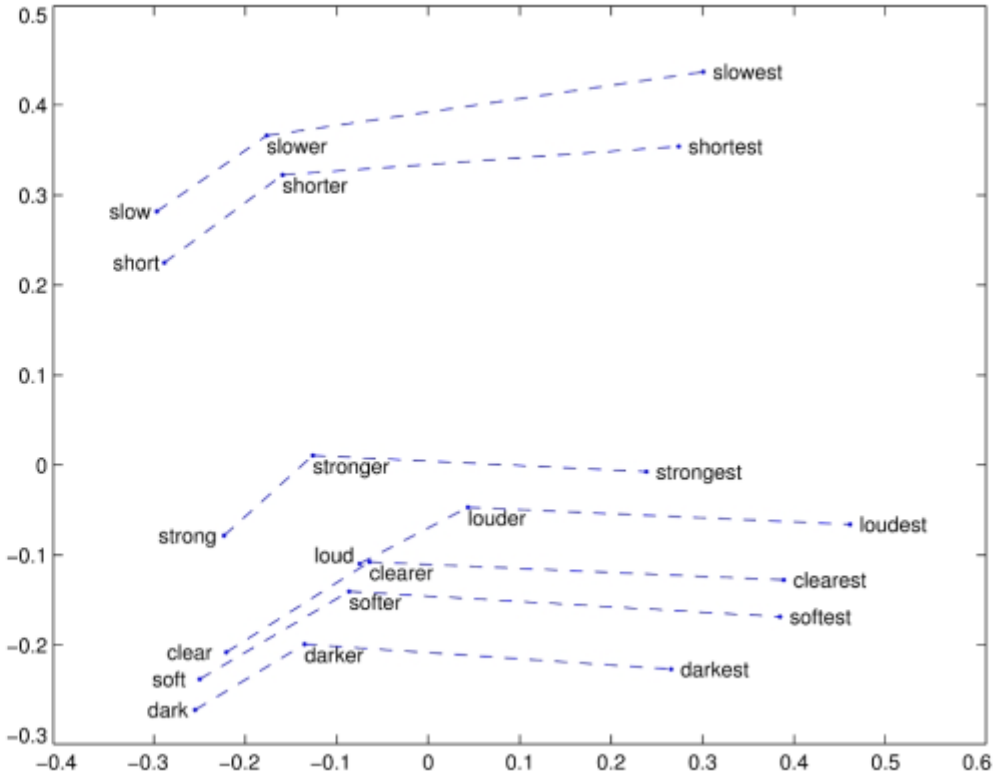
Another very widely used static embedding model is **GloVe,** short for Global Vectors, because the model is based on capturing global corpus statistics. GloVe is based on ratios of probabilities from the word-word cooccurrence matrix, combining the intuitions of count-based models while also capturing the linear structures used by methods like word2vec.

Semantic properties of embeddings

Different types of similarity or association

Analogy/Relational Similarity





$$\overrightarrow{\text{king}} - \overrightarrow{\text{man}} + \overrightarrow{\text{woman}} \text{ is close to } \overrightarrow{\text{queen}}.$$

References

- Daniel Jurafsky and James H. Martin. Speech and Language Processing, 3rd edition, 2025.
- https://medium.com/@shandeep92/countvectorizer-vs-tfidfvectorizer-cf62d0a54fa4
- https://medium.com/@abhishekjainindore24/tf-idf-in-nlp-term-frequency-inverse-document-frequency-e05b65932f1d
- https://www.analyticsvidhya.com/blog/2024/07/tf-idf-matrix/
- https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfTransformer.html#sklearn.feature_extraction.text.TfidfTransformer